

Loafer 多 Agent 软件交付平台

GOAI 世界人工智能开源大赛 · Agent Infra 赛道

选题方向 软件研发全流程协同

团队 [待填写] · 2026-08

问题与场景：企业软件研发的“最后一公里”

- AI 编码助手只能**补全片段**，无法独立完成需求→上线全链路
- 计划、编码、测试、部署、修复跨多个工具与角色，人工切换成本高
- “**Demo 可用、生产难用**”：缺少自动验证、执行证据与风险边界
- 行业普遍场景：重复业务系统构建、内部工具开发、MVP 快速验证等

价值主张：从一句话需求到可访问 URL

- **输入：**自然语言需求（如“做一个带登录的轻量待办应用”）
- **输出：**可直接访问的生产服务 URL
- **中间过程：**计划→模块→任务→代码→测试→修复→部署→验证，全部自动完成
- **人类角色：**设定目标、观察过程、在关键节点确认或兜底

方案总览：基于 AgentTeams 的 Manager-Worker 架构

- **Manager Agent:** 需求理解、计划生成、任务拆解、编排调度、全局决策
- **Worker Agents:** 8 个职能 Agent 各司其职，按依赖串/并行执行
- **协作空间:** 数据库状态机 + SSE 实时事件流，过程透明可审计
- **工具网关:** 统一 SSH/Git/Nginx/Playwright/Shell 接入，凭证不暴露给 Agent
- **人类监督:** Web UI 实时日志、状态面板、暂停/重跑/手动完成

Agent 身份清单：8 个职能 Agent 覆盖全生命周期

Agent	核心职责
计划生成 Agent	需求 → 项目级执行计划
任务分解 Agent	计划 → 带依赖的模块与任务
开发 Agent	按任务步骤编写/修改代码
测试设计 Agent	根据已实现代码生成 API + Web 用例
测试执行 Agent	运行 API / Playwright UI 测试
修复 Agent	读取失败明细，自动改代码
部署 Agent	构建、上传、启动、配置 Nginx
基础架构校验 Agent	验证基础设施模块可构建启动

任务拆解与上下文传递：结构化状态机驱动协同

- **三层拆解**：ExecutionPlan → Module → Task → Step
- **依赖编排**：`sequence\_number` + `blocked\_by`，业务模块严格串行过质量门禁
- **上下文载体**：数据库状态表 + 文件系统产物 + 提示词注入
- **状态流转**：待执行 → 执行中 → 待测试 → 测试中 → 完成 / 测试失败 / 失败

Skill 工程体系：契约维度覆盖评审核验点

契约维度	定义 / 核验点
输入输出	Inputs / Outputs 明确，8 个 Skill 均有
调用条件	何时可被 Agent 调用（如业务模块任务完成后）
依赖工具	Claude CLI / SSH / Playwright / Shell / Nginx 等
失败处理	快速失败 / 降级自由测试 / 修复循环 / 熔断
验证方式	自动测试门禁 / JSON Schema 校验 / 构建启动校验
复用价值	跨项目复用模板、Schema 与执行证据格式
版本演进	语义化版本 v1.0，变更记入 changelog，向后兼容
开源分发设计	Schema + 提示词模板 + 部署脚本分发，Apache-2.0

已落地：internal/engine/skill 注册表 + /projects/:id/available-skills API + 单测校验契约完整性

MCP 与工具集成：等价契约 + 平滑迁移路径

- 当前工具：SSHDeployTool、PlaywrightTool、APIScriptTool、GitTool、NginxTool
- 等价契约已明确：调用入口、Schema、鉴权、重试、幂等、审计、降级
- 迁移到 MCP：将每个工具封装为独立 MCP Server
- 优势：当前设计已完成 Schema 与审计，迁移只需协议适配，无需重写调用链

核心闭环：业务模块自动测试门禁（最多 3 轮）

1. 第 0 步：测试设计 Agent 生成 API / Playwright 用例（幂等）
2. 第 1-3 轮：部署 Agent 全量部署 → 测试 Agent 执行用例
3. 失败 → 修复 Agent 读取失败明细改代码 → 回到第 2 步
4. 通过 → 模块置为完成，继续下一模块
5. 3 轮耗尽 → 流水线暂停，人工确认后断点续跑

可观测、审计与断点续跑：从 Demo 到 Production

- **Trace**: ``claude_call_records`` 记录每次 Agent 调用的 `prompt/output/exit_code/session`
- **Log**: SSE/WebSocket 实时推送 + ``app.log`` / ``backend.log``
- **Metrics**: ``test_run`` 统计通过率/轮次/耗时；部署表统计成功率
- **证据**: Playwright 终态/失败截图、部署记录、测试 JSON
- **断点续跑**: 基于状态机，重启流水线时自动从最近完成点恢复

安全边界与风险控制：高风险动作可审计、可暂停

- 敏感凭证：SSH Key、数据库密码、JWT Secret 由服务层注入，不交给 Agent
- 部署安全：`force` 参数显式控制覆盖式重部署
- 失败熔断：基础架构模块失败立即终止；业务模块失败进入自动修复循环
- 人工兜底：自动测试 3 轮耗尽后暂停；UI 提供“标记完成”跳过异常
- 审计：所有 Agent 调用、工具执行、状态变更持久化到数据库

工程落地可行性：技术栈与运行状态

- 后端：Go 1.21 + Gin + GORM + MySQL（约 1.95 万行 Go 代码）
- 前端：Vue 3 + TypeScript + Element Plus + Vite
- Agent 执行层：Claude Code CLI（--print 离线模式）
- 部署：本地构建 + tar 流上传远程服务器 + Nginx 反向代理
- 当前状态：已实现全链路自动化，完成多个 Demo 端到端交付

当前进展与验证证据：137 次提交，质量闭环已跑通

- 自动用例生成：API + Playwright 场景由测试设计 Agent 自动生成并落库
- 场景级执行：按名匹配回写模块用例，前端展示通过/失败与截图
- 失败自动修复：开发 Agent 读取失败明细，修复后重部署重测
- 截图与报告：终态图、错误图、JSON 结果报告、
`integration-test-report.html`
- 断点续跑：历史空用例模块重启自动补生成，失败模块修复后可恢复

开放与开源计划：从内部平台到可复用 Agent Infra

- 开源 Skill 契约：Agent Identity 模板、输入输出 Schema、失败处理规范
- 开源运行证据格式：测试结果 JSON、截图命名规范、调用记录结构
- 贡献 MCP Server：将部署、测试、Git、Nginx 工具封装为标准 MCP Server
- 社区场景：支持更多技术栈（Python/Java）、更多测试框架、CI/CD 集成
- 许可证：计划采用 Apache 2.0 或 MIT

总结: Loafer = 软件研发领域的 Agent Infra 实践

- 以 AgentTeams Manager-Worker 为设计基点, 8 个 Agent 完成端到端交付
- Skill 抽象 + 等价 MCP 契约, 兼顾当前落地与未来生态兼容
- 自动测试门禁、可观测、审计、断点续跑, 具备 Production 潜力
- 已验证可行, 计划开源, 目标成为企业软件交付的多 Agent 基座

附录：与评分标准对照

评分维度	权重	Loafer 对应内容
场景价值与行业可复制性	25%	软件研发全流程自动化，Go+Vue 业务系统快速构建
多 Agent 协同与自主闭环能力	25%	Manager-Worker 架构、状态机、上下文传递、审批与回滚
Skill 工程体系与生态复用	25%	8 大 Skill 清单、输入输出契约、MCP 迁移路径
工程落地、运行验证与安全可审计	20%	已运行 Demo、测试门禁、截图证据、调用记录、部署日志
开放/开源贡献	5%	计划开源 Skill 契约、运行证据格式、MCP Server

Speaker notes